VULNERABILITY TESTING / LAB REPORT

REFLECTED XSS INTO HTML CONTEXT WITH NOTHING ENCODED

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: PortSwigger Web Security Academy

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	Reflected Cross-Site Scripting (XSS)
Platform	PortSwigger Web Security Academy
Number of Labs	1
Completion Date	25 October 2026

This report documents the objective, vulnerability concept, testing methodology, evidence, impact, remediation, and learning outcomes of the Reflected XSS lab.

Lab 1: Reflected XSS into HTML Context with Nothing Encoded

1. Lab Name

Reflected XSS into HTML Context with Nothing Encoded

Lab URL: https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded

Difficulty: Apprentice
Lab Status: Solved

2. Objective

The objective of this lab is to understand Reflected Cross-Site Scripting (XSS) and identify how user-controlled input can become executable HTML or JavaScript when a web application reflects the input without appropriate output encoding.

The main objectives are:

- Understand reflected XSS.
- Identify user input reflected in an HTML response.
- Examine how special HTML characters are handled.
- Understand how unsafe input handling can lead to JavaScript execution.
- Test the vulnerability in an authorized training environment.
- Learn appropriate XSS prevention and remediation techniques.

3. Vulnerability / Concept

What is Reflected XSS?

Reflected Cross-Site Scripting is a web vulnerability in which an application includes user-controlled input in an HTTP response without safely encoding it for the relevant output context.

If the browser interprets the input as executable script, an attacker may be able to execute JavaScript in the context of the vulnerable application.

How the Vulnerability Works

1. A web application accepts user input, such as a search term.
2. The server includes that input in the returned HTML page.
3. The input is not appropriately encoded.
4. The browser interprets special characters as HTML markup rather than plain text.
5. A crafted input may cause JavaScript to execute in the vulnerable page.

HTML Context

HTML uses special characters to define elements. For example, <h1> and </h1> define a heading.

If an application inserts untrusted input into HTML without proper encoding, the browser may interpret that input as markup. This can allow an attacker to modify the page structure or introduce executable content.

Proof of Concept

The lab demonstrates how a search term can be reflected in the returned page and how unsafe handling of HTML input can lead to reflected XSS.

A common proof-of-concept payload for this lab is:

<script>alert(1)</script>

If the application executes the script in the intended lab context, the browser displays an alert dialog.

Note: This payload is intended for the authorized PortSwigger lab environment. The exact behavior depends on the application's implementation and security controls.

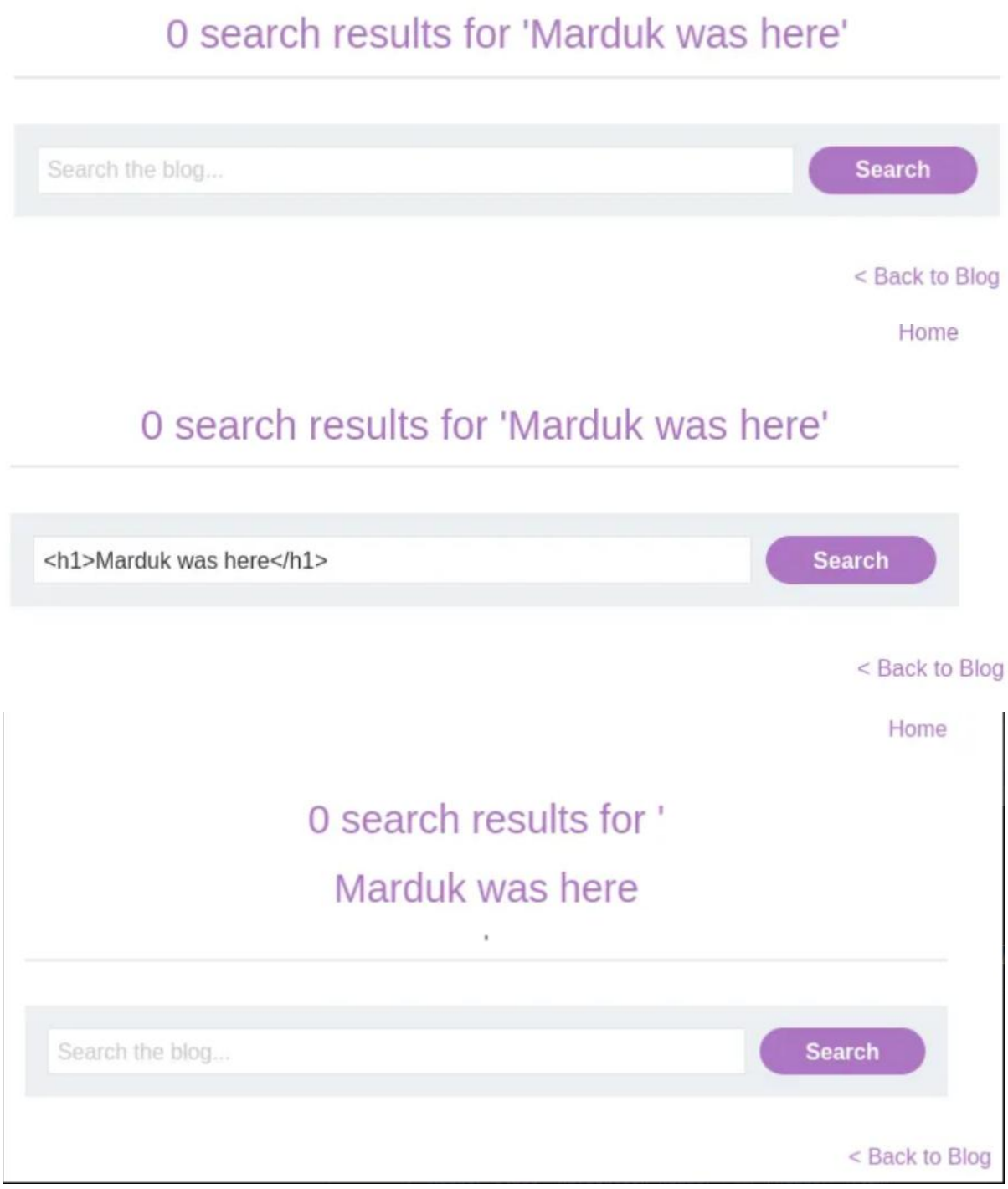
4. Testing Approach

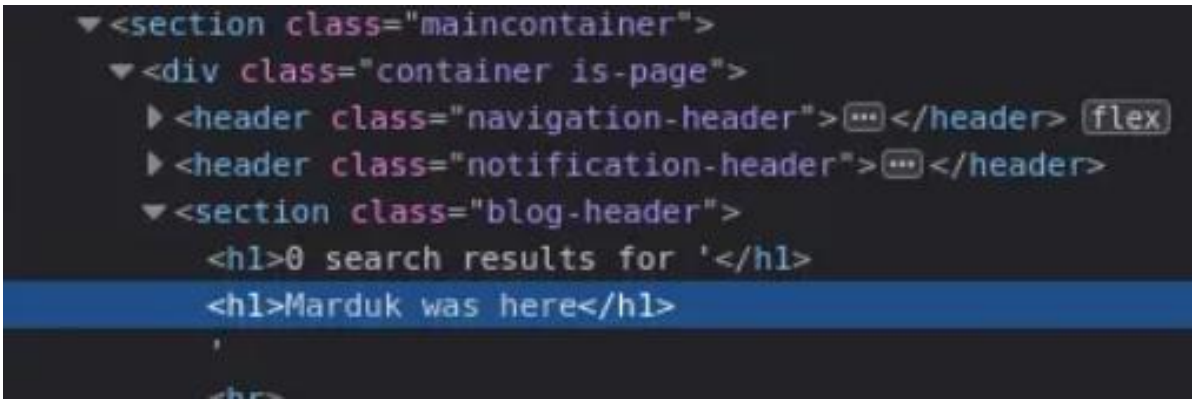
1. Open the Reflected XSS lab in PortSwigger Web Security Academy.
2. Launch the lab environment.
3. Open the provided blog application.
4. Locate the search bar.
5. Enter an ordinary search term.
6. Submit the search and observe whether the term appears in the response.
7. Right-click the reflected search result and select **Inspect**.
8. Examine the HTML element containing the submitted search term.
9. Determine whether the input appears as plain text or is interpreted as markup.
10. Test harmless HTML markup, such as heading tags, within the authorized lab.
11. Observe whether the markup changes the rendered page.
12. Inspect the resulting HTML to determine how the input is reflected.
13. Review the lab's instructions for its required XSS proof of concept.
14. Perform the designated test in the PortSwigger lab.
15. Observe the browser's response to the test.
16. Verify the lab's solved status.
17. Capture screenshots of the relevant behavior and completion state.
18. Record the observed result and potential security impact.
19. Identify suitable output-encoding and secure-rendering controls.
20. Document the findings and remediation recommendations.

Testing note: The supplied screenshot confirms the lab's solved status. Add details about the exact requests or payloads used in your session only if you actually recorded them.

5. Evidence / Screenshots




0 search results for 'Marduk was here'
Search the blog...
Search
< Back to Blog
Home
0 search results for 'Marduk was here'
<h1>Marduk was here</h1>
Search
< Back to Blog
Home
0 search results for '
Marduk was here
'
Search the blog...
Search
< Back to Blog


<section class="maincontainer">

<header class="navigation-header">...</header> flex
<header class="notification-header">...</header>
<section class="blog-header">
<h1>0 search results for '</h1>
<h1>Marduk was here</h1>
'


Home
0 search results for '
Marduk was here
'
<script>alert(document.cookie)</script>
Search
< Back to Blog


ad600f404a6d90e801b67dc00f30011.web-security-academy.net
OK


https://insecure-website.com/search?term=<script>/*+Bad+stuff+here...+*/</
This URL results in the following response:
You searched for: <script>/* Bad stuff here... */</script>
If another user of the application requests the attacker's URL, then the script supplied by the attacker will execute in the victim user's browser, in the context of their session with the application.
LAB
APPRENTICE
Reflected XSS into HTML context with nothing encoded →
Solved
Impact of reflected XSS attacks

6. Impact

If reflected XSS is exploitable in a real application, possible consequences include:

- **Unauthorized actions:** A script may perform actions available to the affected user, subject to application controls.
- **Information exposure:** The script may access information available to JavaScript within the page's security context.
- **Account-related risks:** An attacker may abuse the victim's authenticated application context.
- **Content manipulation:** The page may display misleading or attacker-controlled content.
- **Phishing:** A vulnerable page may be used to present deceptive content.
- **Reputational damage:** Successful exploitation may reduce user trust in the application.

The actual impact depends on the vulnerable context, the victim's privileges, accessible data, and additional browser and server-side protections.

7. Remediation

1. Apply context-aware output encoding

Encode untrusted values appropriately for the output context, such as HTML text or HTML attributes.

2. Use secure templating

Use template engines and frameworks that automatically escape untrusted values where appropriate.

3. Avoid unsafe HTML insertion

When displaying ordinary user-provided text, use safe text-handling APIs instead of APIs that interpret strings as HTML.

4. Validate input appropriately

Validate input according to the application's requirements. Input validation complements output encoding but does not replace it.

5. Sanitize permitted HTML

If users must submit HTML, use a well-maintained HTML sanitizer configured for the application's specific needs.

6. Implement Content Security Policy

A properly configured Content Security Policy (CSP) can reduce the impact of some XSS attacks. It is an additional defense, not a replacement for output encoding.

7. Protect session cookies

Configure cookie attributes such as HttpOnly, Secure, and SameSite according to application requirements. These controls reduce certain risks but do not eliminate the underlying XSS vulnerability.

8. Perform regular security testing

Test search features, URL parameters, error messages, and other areas where user-controlled input may be reflected.

8. What You Learned

The main learning outcomes of this lab are:

- Understanding reflected XSS and how it differs from stored XSS.
- Identifying user-controlled input reflected in an HTML response.
- Understanding why HTML special characters must be handled safely.
- Using browser developer tools to inspect the resulting HTML.
- Understanding how unsafe rendering can lead to JavaScript execution.
- Recognizing the potential impact of XSS on users and their sessions.
- Learning how context-aware output encoding helps prevent XSS.
- Understanding how secure templating and safe DOM APIs improve application security.
- Documenting practical lab completion and evidence for an internship report.

Conclusion

The Reflected XSS lab demonstrates how a web application's search functionality can become vulnerable when user-controlled input is reflected into HTML without appropriate encoding.

The supplied PortSwigger screenshot confirms that the Apprentice-level lab was marked **Solved**. The exercise illustrates the importance of inspecting reflected input, understanding HTML contexts, and validating application behavior during security testing.

Effective prevention requires context-aware output encoding, secure rendering practices, appropriate session protections, and regular web application security testing.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)